

Tipos de Datos y Variables en JavaScript

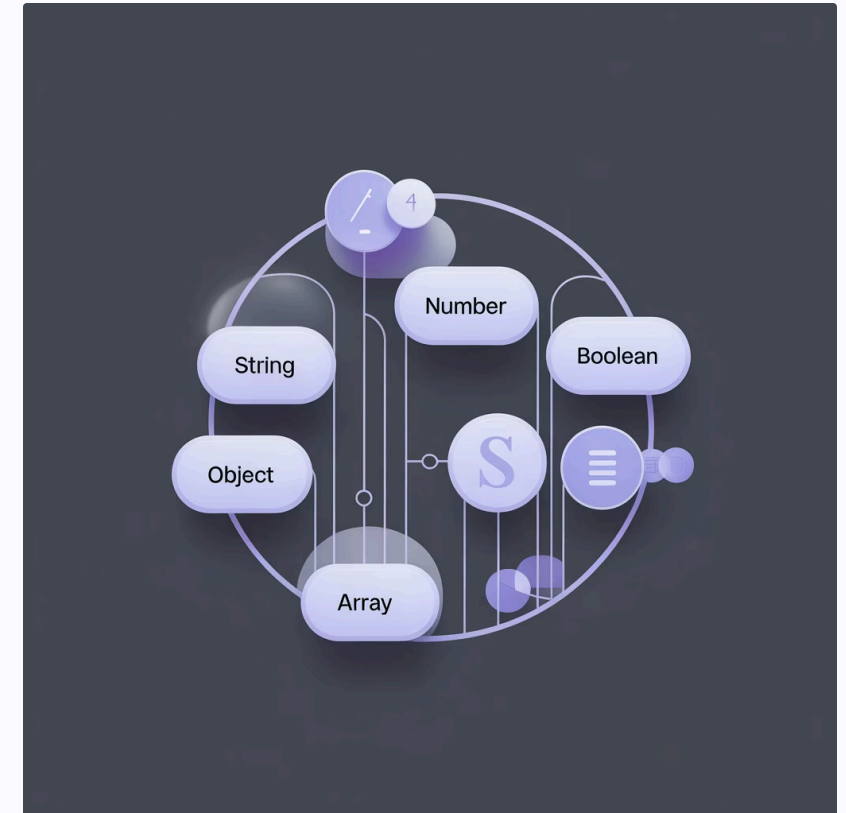
Una guía completa para entender los fundamentos de JavaScript que todo desarrollador necesita dominar



¿Qué son los tipos de datos en JavaScript?

Los tipos de datos en JavaScript definen qué tipo de información puede almacenar y manipular una variable. JavaScript es un lenguaje **dinámicamente tipado**, lo que significa que no necesitas declarar el tipo de una variable explícitamente.

JavaScript clasifica los tipos de datos en dos categorías principales que determinan cómo se almacenan en memoria y cómo se comportan cuando los manipulamos en nuestros programas.



Tipos Primitivos

Valores simples e inmutables que se almacenan por valor

Tipos de Referencia

Estructuras complejas que se almacenan por referencia en memoria

Tipos Primitivos Básicos

Los tipos primitivos son los bloques fundamentales de JavaScript. Son inmutables y se comparan por su valor real.

1

String (Cadena de texto)

Representa texto y se define entre comillas.

```
"Hola mundo"  
'JavaScript es genial'  
`Template literals`
```

2

Number (Número)

Incluye enteros y decimales.
JavaScript no distingue entre ellos.

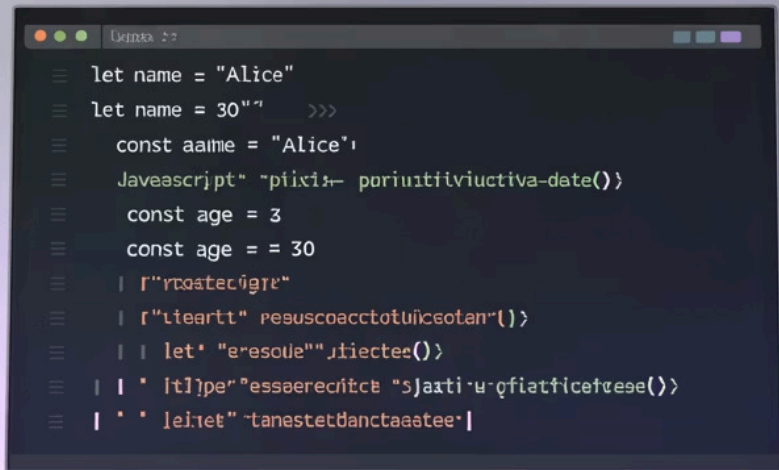
```
42  
3.14159  
-273.15
```

3

Boolean (Booleano)

Representa valores lógicos de verdadero o falso.

```
true  
false
```



```
let name = "Alice"  
let name = 30" >>>  
const aame = "Alice"  
JavaScript "piliis- poruittiviuctiva-date()  
const age = 3  
const age = = 30  
| "roasterügre"  
| "liearti" pesuscoactotuiiceotan{}  
| | let* "eresoule" jtiectee()  
| | " Itl]per" essaerecítce "s]aati-u-qfiatficetveee()  
| | " leiret" tanestetbanctaateet|
```

Tipos Primitivos Especiales

Undefined

Se asigna automáticamente a variables declaradas pero **no inicializadas**.

```
let nombre; // undefined
```

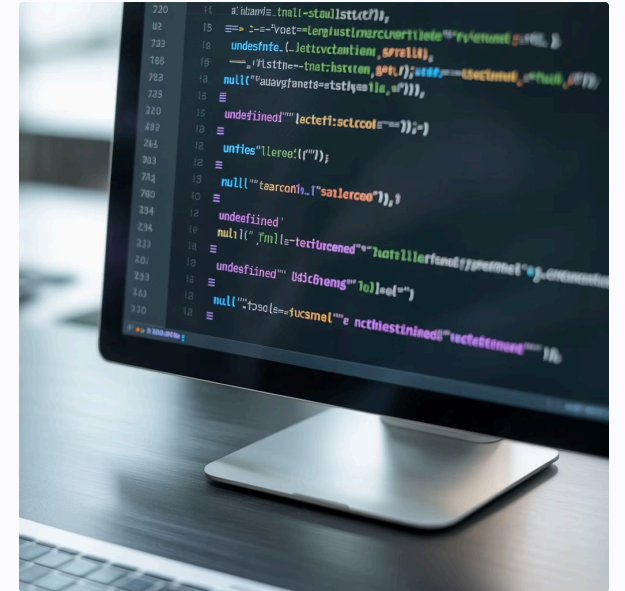
Null

Representa la **ausencia intencional** de cualquier valor.

```
let telefono = null;
```



Característica clave: Los tipos primitivos son inmutables y se comparan por valor. Cuando asignas un primitivo a otra variable, se crea una copia independiente.



Variables: var, let y const

JavaScript ofrece tres formas de declarar variables, cada una con comportamientos diferentes que afectan el alcance y la mutabilidad.

var (Obsoleta)

1

- Alcance de función o global
- Problemas de hoisting
- Permite redeclaración
- **Recomendación:** Evitar su uso

2

let (Recomendada)

- Alcance de bloque
- Para variables que cambian
- No permite redeclaración
- Comportamiento predecible

const (Preferida)

3

- Alcance de bloque
- Debe inicializarse al declarar
- No se puede reasignar
- Contenido mutable en objetos/arrays

Buenas Prácticas para Variables

1 Usa const por defecto

Declara todas las variables con `const` inicialmente. Esto previene reasignaciones accidentales y hace tu código más predecible.

2 let solo cuando sea necesario

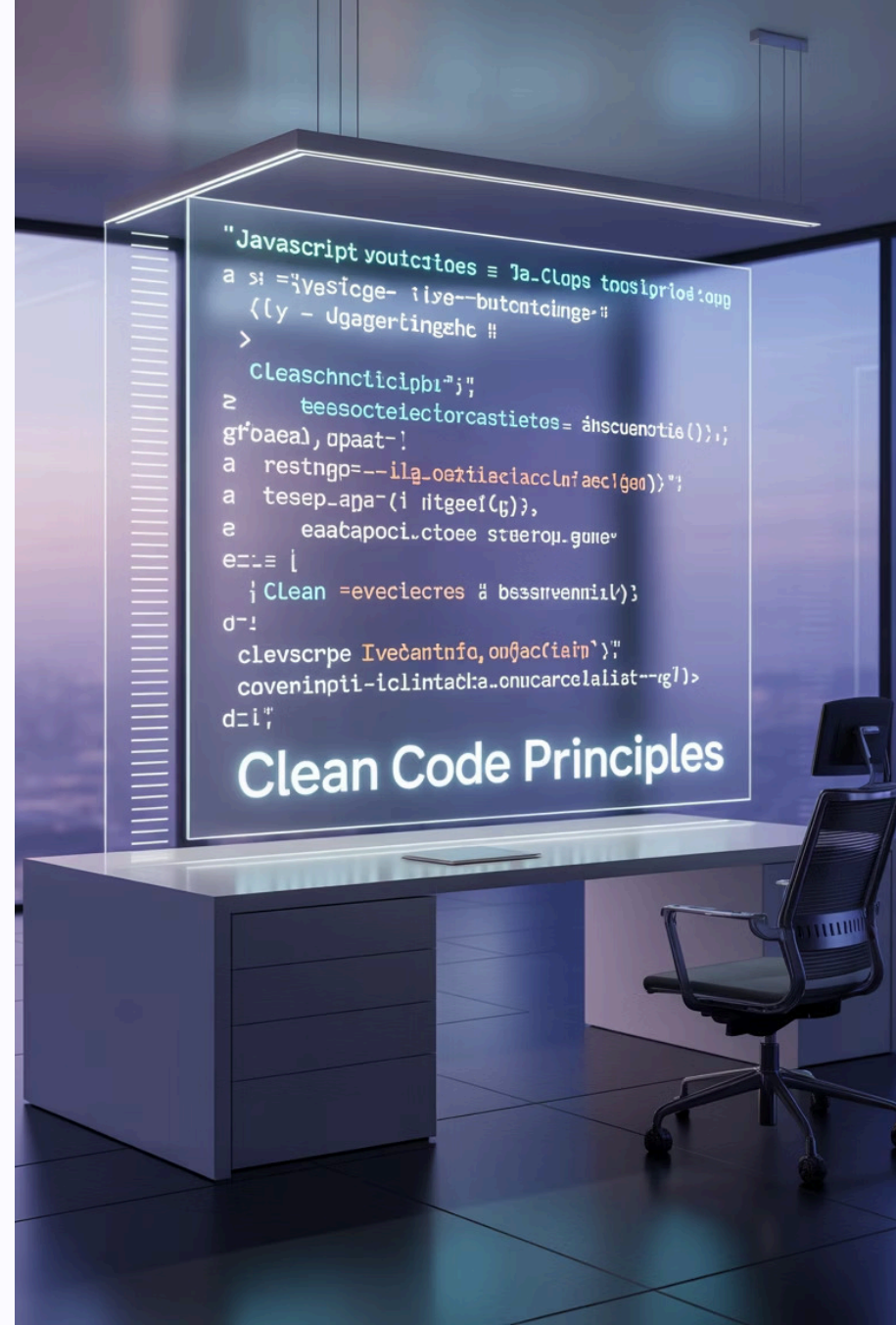
Cambia a `let` únicamente cuando necesites reasignar la variable, como en contadores o acumuladores.

3 Evita var completamente

La palabra clave `var` tiene comportamientos confusos y debe evitarse en código moderno de JavaScript.

4 Nombres descriptivos

Usa nombres que describan claramente el propósito: `usuarioActivo` en lugar de `u` o `activo`.



Ejemplos Prácticos en Código

Veamos cómo aplicar todo lo aprendido en un ejemplo real que muestra todos los tipos de datos y declaraciones de variables.

```
// === Tipos de Datos en JavaScript ===  
// Primitivos  
const nombre = "Alumno";    // string  
let edad = 15;              // number  
let aprobado = true;        // boolean  
let curso;                  // undefined  
let proyecto = null;        // null  
  
// Buenas prácticas  
const PI = 3.14159;         // const para constantes  
let numero = 0;             // let para variables que cambian  
  
// === Mostramos en consola ===  
console.log("Nombre:", nombre);  
console.log("Edad:", edad);  
console.log("¿Está aprobado?", aprobado);  
console.log("Curso:", curso);  
console.log("Proyecto:", proyecto);  
  
console.log("El valor de PI es:", PI);  
  
numero = 5; // cambiamos el valor de una variable con let  
console.log("Nuevo valor de numero:", numero );
```

✓ 💡 **Consejo:** Este ejemplo muestra la aplicación práctica de todos los conceptos. Observa cómo se usan `const` y `let` apropiadamente.

